Unit 11: Bugs

CONTENTS

Objectives
Introduction
11.1 Bug, Error, Fault and Defect
11.2 Classifications of Software Bugs
11.3 Bug Life Cycle
11.4 Bug Tracking
11.5 Bug Tracking Tools
Summery
Keywords
Self Assessment
Answer for Self Assessment
Review Questions
Further Reading

Objectives

After studying this unit, you will be able to:

- discuss bug/defect definition,
- bugs life cycle and bug tracking,
- bug tracking tool

Introduction

Testing is the process of discovering flaws, with a flaw defined as any deviation from expected outcomes. "A coding error is called an Error, a defect discovered by a tester is called a Defect, a defect accepted by the development team is called a Bug, and a build that does not satisfy the requirements is called a Failure."

A software bug is an error, flaw, failure, or fault in a computer programme or system that causes it to generate an inaccurate or unexpected result, or to act in unanticipated ways. A software defect is a difference or deviation in the software application from the end user's or original business requirements. A software defect is a coding fault that results in inaccurate or unexpected outputs from a software programme that does not satisfy its intended purpose. During the execution of test cases, testers may come across such flaws.

11.1 Bug, Error, Fault and Defect

A bug is caused by a code mistake. Before the product is shipped to the customer, an error was discovered in the development environment. An error in programming that causes a programme to perform poorly, generate inaccurate results, or crash. A malfunctioning programme caused by a software or hardware fault. The term "bug" is used by testers.

A "Bug" is a most unwelcomed word in the software development process. A bug indicates a fault, error or failure in the software/system being built that produces unexpected results. A bug identified needs to be tracked and fixed to ensure optimum quality in the software/system being developed.

An error in a computer program's step, process, or data specification that leads the programme to behave in an unexpected or unexpected way. As a result of an error, a flaw is introduced into the software. It's a flaw in the software that could lead it to behave erroneously and in ways that aren't specified. It's the outcome of a mistake. The software industry is still unable to agree on definitions for all of the aforementioned terms. In other words, if you use the term to signify one thing, your audience may not understand it to mean that thing.

When the predicted and actual behaviour do not match in software testing, an incident must be raised. An occurrence could be a Bug. When a programmer intends to implement a given behaviour, but the code fails to correctly comply to that behaviour due to faulty coding implementation, it is the programmer's fault. It's also known as a flaw.

An error is a mistake, misconception, or misunderstanding on the part of a software developer. In the category of the developer, we include software engineers, programmers, analysts, and testers. For example, a developer may misunderstand a de-sign notation, or a programmer might type a variable name incorrectly – leads to an Error. It is the one that is generated because of the wrong login, loop or syntax. The error normally arises in software; it leads to a change in the functionality of the program.

Bugs in history and their impact

In 1996, the $1.0 billion rocket called Ariane 5 was destroyed a few seconds after launch due to a bug in the on-board guidance computer program.

In 1962, a bug in the flight software for the Mariner I spacecraft caused the rocket to change path from the expected path.

In the 1980s, bugs in the code controlling the machine called Therac-25, used for radiation therapy, lead to patient deaths.

In the 1990s, a bug was found in the new release of AT&T's software control #4ESS long distance switches caused many computers to crash.

Bugs and its Source

A bug is actually an error which would have been introduced in the due course of the software development life cycle. The most common sources of Bugs are detailed below.

Ambiguous Requirements – Unclear requirement is a source of Bugs being introduced. This can be avoided by having strong Static Testing techniques, Reviews, Walkthroughs, etc.

Programming Errors – Incorrect coding standards is a source which can be minimized with proper code review and unit testing

Unachievable Deadlines – Stringent timelines often lead to bugs which can be avoided if proper planning is done to ensure timelines are driven based on the estimates

Missing coding - Here, missing coding means that the developer may not have developed the code only for that particular feature.

Every bug not only breaks the functionality it occurs in but also has probabilities of causing a ripple effect in other areas of the application. This ripple effect needs to be evaluated when fixing any bug. Lack of foresight in anticipating such issues can cause serious problems and an increase in bug count.

Bugs in Software Testing and Its Impact

The impact of the Bug is measured based on Severity and Priority. Impact to business is measured in terms of Severity, whereas impact to business execution is measure in terms of Priority.

The table below is a standard definition used across the software industry for the Severities.

Severity	Definition
Critical	A critical defect would create a major disruption to the business operation. A severe application problem causing considerable downtime, financial penalty or loss of integrity with customers.
High	A major defect would result in loss of business functionality and would require a workaround in production.

Medium	A defect which would have a medium impact on business functions, but could be immediately managed or worked around.
Low	A cosmetic only defect which would have no business or user impact

Priority determines the impact of business execution, and subsequently, how quickly the defect needs to be fixed to ensure the project plan is on track.

The table below is a standard definition used across the software industry for the Priorities.

Priority	Definition
Immediate	Any problem that is stopping ALL usage of the software by the business, i.e. Nothing else can be executed until this problem has been corrected, tested and re-migrated with No workaround**.**
High	A major function or feature has been disabled or is incorrect causing a severe degradation in service. A workaround is possible, but additional problems/failures could result in critical failure.
Medium	An issue which signifies one of the multiple streams of testing (other than the critical path) is stopped, but other streams of processing can still continue.
Low	Business can still proceed with a workaround and the fix can be applied in isolation

11.2 Classifications of Software Bugs

Software defects by nature

- Functional defects
- Performance defects
- Usability defects
- Compatibility defects
- Security defects

Functional defects are the errors identified in case the behaviour of software is not compliant with the functional requirements. Such types of defects are discovered via functional testing

Performance defects are those bound to software's speed, stability, response time, and resource consumption, and are discovered during performance testing.

Usability defects make an application inconvenient to use and, thus, hamper a user's experience with software.

Compatibility defects: An application with compatibility errors doesn't show consistent performance on particular types of hardware, operating systems, browsers, and devices or when integrated with certain software.

Security defects are the weaknesses allowing for a potential security attack. The most frequent security defects in projects we perform security testing for are encryption errors, susceptibility to SQL injections, XSS vulnerabilities, buffer overflows, weak authentication, and logical errors in role-based access.

Software defects by severity

- Critical defects
- High-severity defects
- Medium-severity defects
- Low-severity defects

Critical defects usually block an entire system's or module's functionality, and testing cannot proceed further without such a defect being fixed.

An example of a critical defect is an application's returning a server error message after a login attempt.

High-severity defects affect key functionality of an application, and the app behaves in a way that is strongly different from the one stated in the requirements, for instance, an email service provider does not allow adding more than one email address to the recipient field.

Medium-severity defects are identified in case a minor function does not behave in a way stated in the requirements.

Low-severity defects are primarily related to an application's UI and may include such an example as a slightly different size or color of a button.

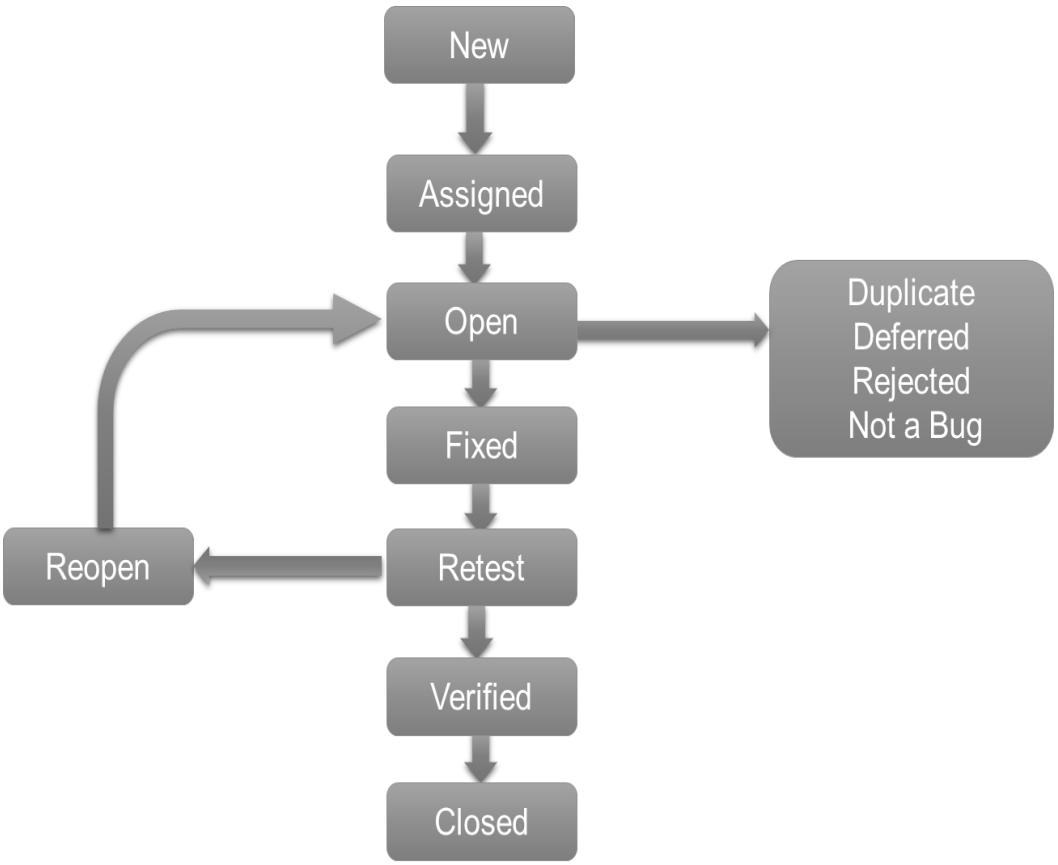
11.3 Bug Life Cycle

Bug Life Cycle is the specific set of states that defect or bug goes through in its entire life.

This starts as soon as any new defect is found by a tester and comes to an end when a tester closes that defect assuring that it won't get reproduced again.

Bug Life cycle is the journey of a defect cycle, which a defect goes through during its lifetime. It varies from organization to organization and also from project to project as it is governed by the software testing process and also depends upon the tools used.

Bug life cycle



New: This is the first state of a defect in the Defect Life Cycle. When any new defect is found, it falls in a 'New' state, and validations and testing are performed on this defect in the later stages of the Defect Life Cycle.

Assigned: In this stage, a newly created defect is assigned to the development team for working on the defect. This is assigned by the project lead or the manager of the testing team to a developer.

Open: Here, the developer starts the process of analyzing the defect and works on fixing it. if required. If the developer feels that the defect is not appropriate then it may get transferred to any of the below four states namely Duplicate, Deferred, Rejected, or Not a Bug-based upon the specific reason.

Fixed: When the developer finishes the task of fixing a defect by making the required changes then he can mark the status of the defect as 'Fixed'.

Pending Retest: After fixing the defect, the developer assigns the defect to the tester for retesting the defect at their end, and till the tester works on retesting the defect, the state of the defect remains in 'Pending Retest'.

Retest: At this point, the tester starts the task of working on the retesting of the defect to verify if the defect is fixed accurately by the developer as per the requirements or not.

Reopen: If any issue persists in the defect then it will be assigned to the developer again for testing and the status of the defect gets changed to 'Reopen'.

Verified: The tester re-tests the bug after it got fixed by the developer. If there is no bug detected in the software, then the bug is fixed and the status assigned is "verified."

Closed: When the defect does not exist any longer then the tester changes the status of the defect to 'Closed'.

Rejected: If the defect is not considered as a genuine defect by the developer then it is marked as 'Rejected' by the developer.

Duplicate: If the developer finds the defect as same as any other defect or if the concept of the defect matches any other defect then the status of the defect is changed to 'Duplicate' by the developer.

Deferred: If the developer feels that the defect is not of very important priority and it can get fixed in the next releases or so in such a case, he can change the status of the defect as 'Deferred'.

Not a Bug: If the defect does not have an impact on the functionality of the application then the status of the defect gets changed to 'Not a Bug'.

Guidelines for Implementing Defect/ bug Life Cycle

- It is very important that before starting to work on the Defect Life Cycle, the whole team clearly understands the different states of a defect (discussed above).
- Defect Life Cycle should be properly documented to avoid any confusion in the future.
- Make sure that each individual who has been assigned any task related to the Defect Life Cycle should understand his/her responsibility very clearly for better results.
- Each individual who is changing the status of a defect should be properly aware of that status and should provide enough details about the status and the reason for putting that status so that everyone who is working on that particular defect can understand the reason of such a status of a defect very easily.
- The defect tracking tool should be handled with care to maintain consistency among the defects and thus, in the workflow of the Defect Life Cycle.

11.4 Bug Tracking

During software testing, bug tracking is the process of logging and monitoring defects or errors. It's also known as issue tracking or defect tracking. Hundreds or thousands of flaws may exist in large systems. For debugging, each must be reviewed, monitored, and prioritized. Bugs may need to be monitored over a long period of time in some circumstances. An effective bug tracking system that is perfectly aligned with your development and quality processes is a priceless asset.

defect tracking is a vital step in software engineering because complicated and business-critical systems have hundreds of flaws. "Managing, evaluating, and prioritising these issues is one of the most difficult aspects. Over time, the number of defects grows, and to successfully manage them, a defect tracking system is utilised to make the task easier.

A bug tracking tool can help record, report, assign and track the bugs in a software development project. There are many defect tracking tools available. You can put this in another way "Better is the bug tracking tool, better the quality of the product."

For Bug tracking software, it is essential to have:

Reporting facility – complete with fields that will let you provide information about the bug, environment, module, severity, screenshots, etc.

Assigning – What good is a bug when all you can do is find it and keep it to yourself, right?

Progressing through life cycle stages – Workflow

History/work logs/comments

Reports – graphs or charts

Storage and Retrieval – Every entity in a testing process needs to be uniquely identifiable. The same rule applies to bugs too. A bug tracking tool must provide a way to have an ID that can be used to store, retrieve (search) and organize bug information.

Functionalities of the Bug Tracking system is:

Creating a new text file and writing the details entered by the user into the text file.

Option to change the status of the bug.

Report of specific bug file.

11.5 Bug Tracking Tools

Following is a list of Top Bug Tracking Tool, with their popular features. The list contains both open source (free) and commercial (paid) software.

BugZilla	BackLog
SpiraTeam	Bird Eats Bug
Zoho bug tracker	Monday
JIRA	Mantis
RedMine	Trac
Axosoft	HP ALM/ Quality Center
eTraxis	Bugnet

BugZilla:

Bugzilla has been a leading Bug Tracking tool widely used by many organizations for quite some time now. It is very simple to use, a web-based interface. It has all the features of the essence, convenience, and assurance.

This tool is an open source software and provides features like

- E-mail notification for change in code
- Reports and Charts
- Patch Viewers
- List of bugs can be generated in different formats
- Schedule daily, monthly and weekly reports
- This bug tracking tool detect duplicate bug automatically
- Setting bug priorities by involving customers
- Predict the time a bug may get fixed

Backlog:

Backlog is a popular bug and project tracking tool in one platform. It's easy for anyone to report bugs and keep track of a full history of issue updates and status changes.

- Easy bug tracking tool
- Search and advanced search features

- Full history of issue updates and status changes
- Project and issues with subtasks
- Git and SVN built-in
- Gantt Charts and Burndown charts
- Wikis and Watchlists
- Native mobile apps
- Kanban-style boards for visual workflow

SpiraTeam:

SpiraTeam provides a complete Application Lifecycle Management (ALM) solution that manages the requirements, tests, plans, tasks, bugs and issues in one environment, with complete traceability.

- Automatic creation of new incidents during the test script execution.
- Fully customizable incident fields including statuses, priorities, defect types and severities
- Ability to link incidents (bugs) to other artifacts and incidents.
- Robust reporting, searching and sorting, plus an Audit log tracking changes.
- Email notifications triggered by the customized workflow status changes.
- Ability to report issues and bugs via email.

Bird Eats Bug:

Bird Eats Bug is a bug reporting tool that helps anyone create interactive data-rich bug reports. While a user makes a screen recording of the issue, Bird's browser extension automatically augments it with valuable technical data like console logs, network errors, browser information, etc.

- Each report auto-includes console logs, network requests and general info (browser/OS, etc.)
- Use mic and/or video recording to explain actions. Save time on typing descriptions.
- Integrations with issue trackers like JIRA, Trello, Github, etc.

Zoho bug tracker:

Zoho bug tracker is a powerful bug tracker that helps you to view issues filtered by priority and severity. It improves the productivity by exactly knowing which bugs are reproducible. It is an online tool that allows you to create projects, bugs, milestone, reports, documents, etc. on a single platform.

Monday:

Monday is a bug tracking tool that that enables you to analyze your performance and manage your team in one place. It provides flexible dashboard for easy visualization of data.

- You can collaborate with other people.
- It can automate your daily work.
- Integration with Mailchimp, Google calendar, Gmail, and more.
- You can track your work progress.
- It enables you to work remotely.

JIRA:

It is easy to use framework. JIRA is a commercial product and helps to capture and organize the team issues, prioritizing the issue and updating them with the project.

It is a tool that directly integrates with the code development environments making it a perfect fit for developers as well.

It comes with many add-ons that make this tool more powerful.

Mantis:

Mantis comes as a web application but also has its own mobile version. It works with multiple databases like MySQL, PostgreSQL, MS SQL and integrated with applications like chat, time tracking, wiki, RSS feeds and many more.

- This is an Open source tool
- This defect tracking tool provides E-mail notification
- Supported reporting with reports and graphs
- Source control integration
- Supports custom fields
- Supports time tracking management

- Multiple projects per instance
- Enable to watch the issue change history and roadmap
- Supports unlimited number of users, issues, and projects

RedMine:

It is an open source bug tracking tool that integrates with SCM (Source Code Management System) too.

It supports multiple platforms and multiple data-bases while for reporting purpose, Gantt charts and calendar are used.

- Gantt chart and calendar
- News, document and files management
- This bug reporting tool provides SCM integration
- Issue creation via e-mail
- This bug tracking software supports multiple database.
- Flexible issue tracking system
- Flexible role based access control
- Multilanguage support

Trac:

Trac is a web based open source issue tracking system that developed in Python.

It is used to browse through the code, view history, view changes, etc. when you integrate Trac with SCM.

It supports multiple platforms like Linux, Unix, Mac OS X, Windows, etc.

Axosoft:

It is a bug tracking system, available for hosted or on-premises software. It is a project management tool for Scrum teams.

It can view each task, its requirement, defects and incidents, in the system, on individual filing cards, through the Scrum planning board.

Axosoft can manage your user stories, defects, support tickets and a real-time snapshot of your progress.

- Scrum planning board
- Scrum burn down charts
- Requirement Management
- Team wiki
- Data visualization
- SCM integration
- Reporting
- Help desk or incident tracking

HP ALM/ Quality Center:

HP ALM is a complete test management solution with a robust integrated bug tracking System within it. HP ALM's bug tracking mechanism is easy and efficient. It supports Agile projects too.

eTraxis:

eTraxis is an open source bug tracking tool supporting multiple languages. This tool is developed in a PHP language and supports multiple-database like Oracle, MySQL, PostgreSQL, and MS Server.

eTraxis gives you a flexible platform involving multiple organizations by providing a central place for all project activity. It allows to create multiple users and projects and at the same time view the bugs assigned.

- Files exchange and supports attachment
- E-mail notification
- Flexible permission
- Powerful filtering on issues
- Custom workflow
- View complete history of all events

IBM Rational ClearQuest:

- IBM ClearQuest tracks, captures and manages any type of bugs
- It support multiplatform, like HP-UX, Linux, Microsoft Windows operating system. It can improve visibility and control of software development projects.
- Integrating with other tools
- Supports real time reporting and metrics
- Increasing team collaboration

Summery

A Software Bug is a failure or flaw in a program that produces undesired or incorrect results. It's an error that prevents the application from functioning as it should.

Reasons For Software Bugs: Miscommunication or No Communication

Software Complexity

Programming Errors

Changing Requirements

Poorly Documented Code

A Bug tracking system is also called as a defect tracking system, this is a software application that captures, reports, and manages data on bugs (errors, exceptions) that occur in software.

Keywords

Bug ***Error***

Fault ***Defect***

Bug tracking

Bug tracking tools

Self Assessment

1. What are the reasons for bug?

A. Wrong coding
B. Missing coding
C. Extra coding
D. All of above

2. What are the classifications of software bugs?

A. Software defects by priority
B. Software defects by severity
C. Software defects by nature
D. All of above

3. Performance defects and Usability defects are part of___

A. Software defects by severity
B. Software defects by nature
C. Software defects by priority
D. None of above

4. Software defects by priority consist of __

A. Critical defects
B. High-severity defects
C. Medium-severity defects
D. None of above

5. Software defects by priority are based upon_____
A. Severity
B. Priority
C. By nature
D. All of above

6. Which is not part of Bug life cycle states?
A. Open
B. Fixed
C. Design
D. Retest

7. Assign, fixed and retest is part of___
A. Software defects by priority
B. Bug life cycle
C. Software defects by nature
D. All of above

8. Which is not bug tracking tool?
A. Zoho tracker
B. Bootstrap
C. Monday
D. JIRA

9. Trac is developed in_
A. Java
B. C++
C. Python
D. None of above

10. eTraxis support___
A. MySQL
B. PostgreSQL
C. MS Server
D. All of above

11. Bug tracking system is__
A. Report of specific bug file
B. It is process of logging and monitoring bugs
C. Option to change the status of the bug
D. All of above

12. Which is not Bug life cycle state?
A. Open
B. Feasibility study
C. Reset
D. None of above

13. eTraxis, Bugnet and Axosoft are__
A. Design tool
B. Testing tools
C. Bug tracking tool
D. None of above

14. Mantis is___
A. Open source tool
B. Bug tracking tool
C. Works with multiple databases like MySQL, PostgreSQL
D. All of above

15. Gantt chart and calendar are used in_
A. Trac
B. RedMine
C. JIRA
D. All of above

Answer for Self Assessment

1.	D	2.	D	3.	B	4.	D	5.	B
6.	C	7.	B	8.	B	9.	C	10.	D
11.	D	12.	B	13.	C	14.	D	15.	B

Review Questions

1. Define bug with example.
2. What are the reasons for bug?
3. Differentiate between bug, error, and fault.
4. Explain any two types of bug.
5. Discuss bug priority and severity.
6. What is significance of bug tracking?
7. Discuss any four bug tracking tools.
8. What are the major features of bug tracing tools?

Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner's Approach, 5th Edition, Tata
- McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education

Online Links

- www.guru99.com
- www.ece.cmu.edu
- www.softwarequalitymethods.com
- https://www.softwaretestinghelp.com/popular-bug-tracking-software
- https://www.atlassian.com/software/jira/features/bug-tracking
- https://www.softwaretestingmaterial.com/popular-defect-tracking-tools/
- https://www.javatpoint.com/defect-or-bug-tracking-tool